

Sprint05_MongoDB

Yonadxandi Manriquez

Nivel 1

Crea una base de datos con MongoDB utilizando como colecciones los archivos adjuntos.

Para trabajar con MongoDB en este sprint, el proceso fue el siguiente:

Primero se creó una cuenta en MongoDB Atlas, que es la plataforma en la nube de MongoDB. Dentro de Atlas se creó un clúster gratuito llamado Cluster0, usando AWS como proveedor y París (eu-west-3) como región, por ser la más cercana a España.

Se intentó trabajar directamente desde la interfaz web de Atlas para importar los archivos JSON, pero Atlas web no permite importar archivos JSON completos de forma sencilla, solo permite insertar documentos uno a uno. Esta limitación obligó a buscar otra solución.

La solución fue instalar **MongoDB Compass**, que es la aplicación de escritorio oficial de MongoDB. Compass se conecta al clúster de Atlas mediante una cadena de conexión del tipo:

```
mongodb+srv://usuario:contraseña@cluster0.xxx.mongodb.net/
```

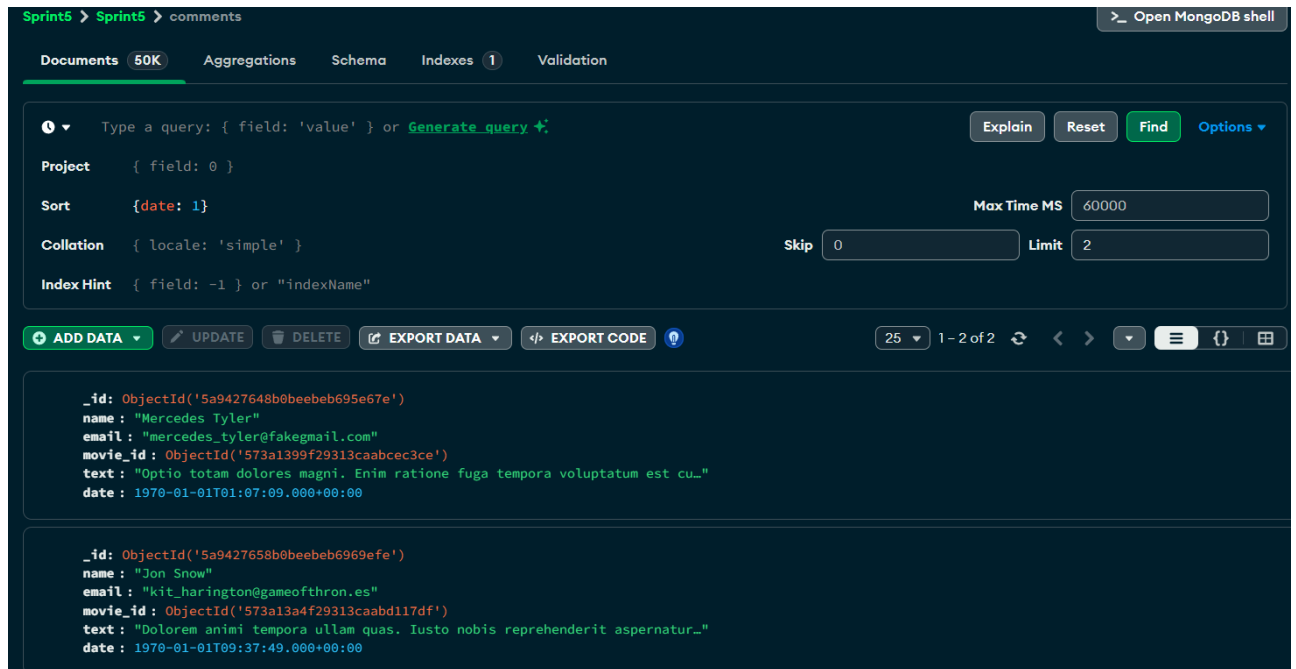
Compass se eligió porque permite importar archivos JSON completos en segundos.

Una vez conectada Compass al clúster, se creó la base de datos Sprint5 con 5 colecciones importando cada archivo JSON:

- comments
- movies
- theaters
- users
- sessions

Ejercicio 1

Muestra los 2 primeros comentarios que hay en la base de datos.



The screenshot shows the MongoDB Compass interface for the 'comments' collection. The query is configured to sort by 'date' in ascending order and limit the results to 2 documents. The results show two comments:

```
{
  "_id": ObjectId("5a9427648b0beebe695e67e"),
  "name": "Mercedes Tyler",
  "email": "mercedes_tyler@fakegmail.com",
  "movie_id": ObjectId("573a1399f29313caabcec3ce"),
  "text": "Optio totam dolores magni. Enim ratione fuga tempora voluptatum est cu-",
  "date": 1970-01-01T01:07:09.000+00:00
}
```

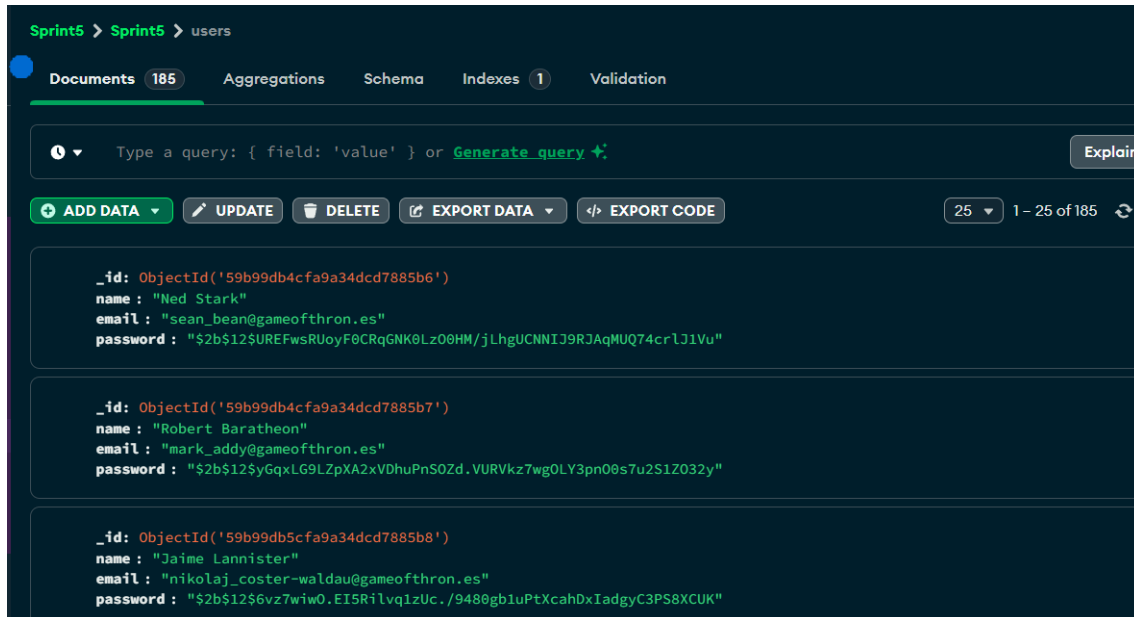
```
{
  "_id": ObjectId("5a9427658b0beebe6969efe"),
  "name": "Jon Snow",
  "email": "kit_harington@gameofthron.es",
  "movie_id": ObjectId("573a13a4f29313caabd117df"),
  "text": "Dolorem animi tempora ullam quas. Iusto nobis reprehenderit aspernatur-",
  "date": 1970-01-01T09:37:49.000+00:00
}
```

Se consultó la colección comments sin filtro, ordenando por fecha de forma ascendente `date: 1` para obtener los comentarios más antiguos, y se limitó el resultado a 2 documentos.

El análisis de los comentarios más antiguos permite identificar el comportamiento de los primeros usuarios de la plataforma. Comparando estos perfiles iniciales con los actuales, es posible detectar si el tipo de usuario ha evolucionado con el tiempo, lo que puede orientar decisiones sobre posicionamiento de producto y segmentación histórica de la audiencia.

Los 2 primeros comentarios por fecha son de Mercedes Tyler y Jon Snow.

¿Cuántos usuarios tenemos registrados?



Se puede apreciar a un lado de Documents sin necesidad de hacer ninguna consulta, el número es 185.

Con la consulta construida en el Shell aparece:

```
> db.users.countDocuments()
< 185
```

Se utilizó countDocuments() sobre la colección users para contar el total de documentos, donde cada documento representa un usuario único registrado en la plataforma.

Esta consulta en términos de negocio, nos ayuda a saber la tasa de comentarios por usuario. También, se puede analizar cuántos usuarios están activos para después realizar campañas.

¿Cuántos cines hay en el estado de California?

```
> db.theaters.countDocuments({"location.address.state": "CA"})
< 169
```

Se filtró la colección theaters usando el campo anidado location.address.state con el valor "CA" (código de

California). En MongoDB los campos anidados se acceden con notación de punto.

En este caso, al conocer la concentración de cines de forma geográfica, permite realizar estrategias comerciales, por ejemplo, identificar mercados con baja presencia.

¿Cuál fue el primer usuario/a en registrarse?

```
> db.users.find().sort({ _id: 1 }).limit(1)
< {
  _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
  name: 'Ned Stark',
  email: 'sean_bean@gameofthron.es',
  password: '$2b$12$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNIJ9RJAqMUQ74cr\JJ1Vu'
}
```

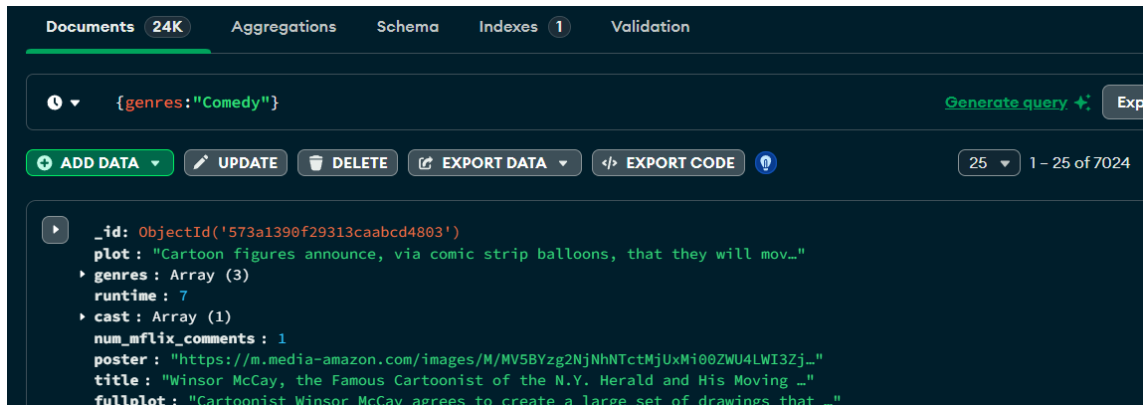
Se ordenó la colección users por el campo _id de forma ascendente { _id: 1 }. En MongoDB, el _id es un ObjectId que contiene la fecha de creación codificada dentro, por lo que ordenar por _id: 1 equivale a ordenar por fecha de registro de más antiguo a más reciente. Con limit(1) obtenemos solo el primero.

¿Por qué es importante saberlo? Podemos reconstruir históricos, saber cuando empezamos con las campañas y analizarlas, para después atraer mas usuarios.

¿Cuántas películas de comedia hay en nuestra base de datos?

```
> db.movies.countDocuments({ genres: "Comedy" })
< 7024
```

Se filtró la colección movies usando el campo genres con el valor "Comedy" obteniendo como resultado 7024. También se consulto a través de la colección.

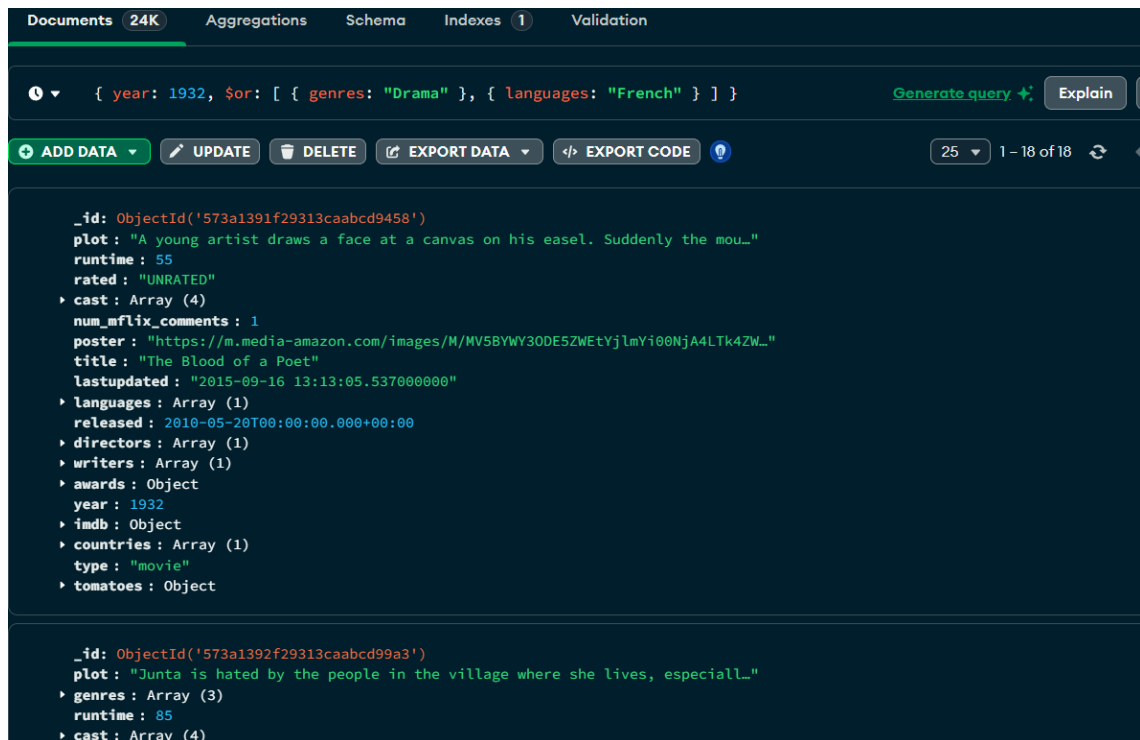


Saber que la comedia es uno de los géneros más representados permite al equipo de contenidos priorizar acuerdos de licencia y campañas temáticas.

Ejercicio 2

Muéstrame todos los documentos de las películas producidas en 1932, pero que el género sea drama o estén en francés.

```
> db.movies.countDocuments({ year: 1932, $or: [ { genres: "Drama" }, { languages: "French" } ] })
< 18
```



Se usaron dos condiciones combinadas. El campo `year: 1932` es obligatorio para todos los resultados. Dentro del `$or` se busca que `genres` contenga "Drama" o que `languages` contenga "French".

De todas las películas producidas en 1932, solo 18 cumplen la condición de ser drama o estar en francés.

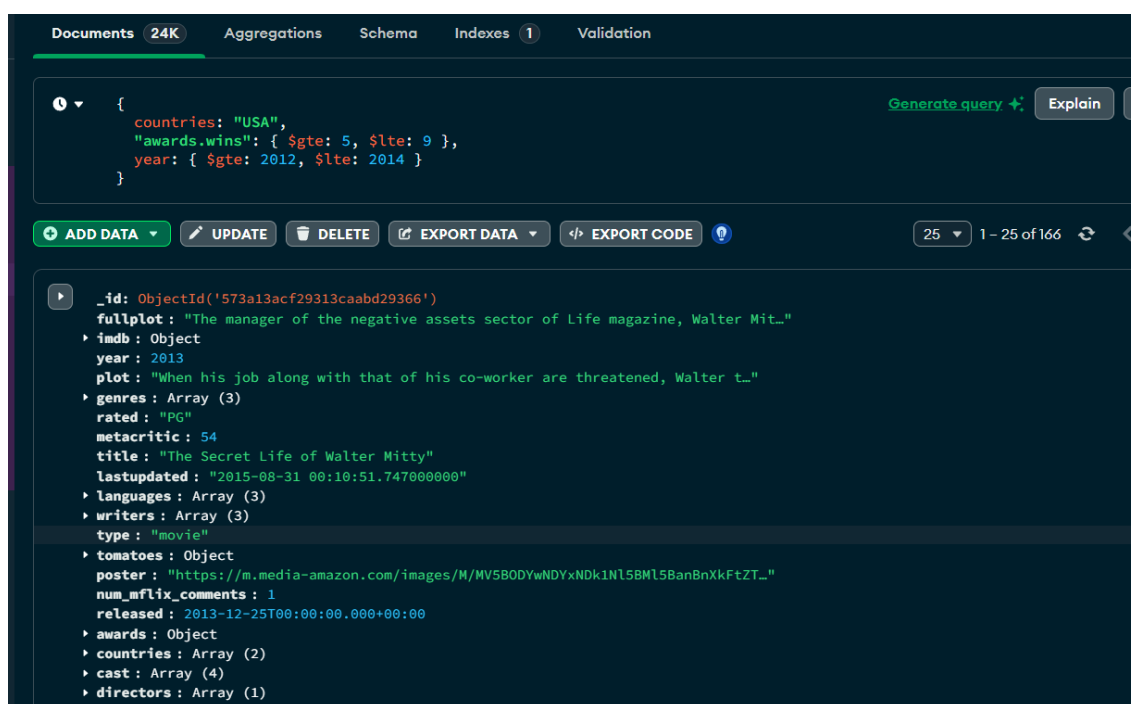
En el caso concreto del cine francés de 1932, este tipo de consulta puede alimentar secciones de la plataforma orientadas a públicos con interés en cine de autor o patrimonio cinematográfico europeo.

Ejercicio 3

Muéstrame todos los documentos de películas estadounidenses que tengan entre 5 y 9 premios que fueron producidas entre 2012 y 2014.

En MongoDB no existe BETWEEN como en SQL, por eso usamos \$gte y \$lte juntos para filtrar los premios ganados y el intervalo de años.

```
> db.movies.countDocuments({ countries: "USA", "awards.wins": { $gte: 5, $lte: 9 }, year: { $gte: 2012, $lte: 2014 } })
< 166
```



Se usaron tres filtros simultáneos. `countries: "USA"` filtra por país. `"awards.wins"` accede al campo anidado dentro del objeto `awards` usando notación de punto, y con `$gte` y `$lte` definimos el rango de 5 a 9 premios. Lo mismo para `year` con el rango 2012-2014. Al estar todos al mismo nivel dentro de `{}` funcionan como un AND donde todas las condiciones deben cumplirse.

Se podría haber optamos por un `$or` entre los premios y el año, pero esto hubiese sido un error lógico, convirtiendo las condiciones en opcionales y no obligatorias como muestra el ejercicio.

Identificar películas estadounidenses galardonadas dentro de un rango de años específico es una herramienta directa para el equipo de adquisición de contenidos.

Las películas con entre 5 y 9 premios representan un nivel de reconocimiento, lo que las convierte en candidatas ideales para acuerdos de licencia para futuras proyecciones.

Nivel 2

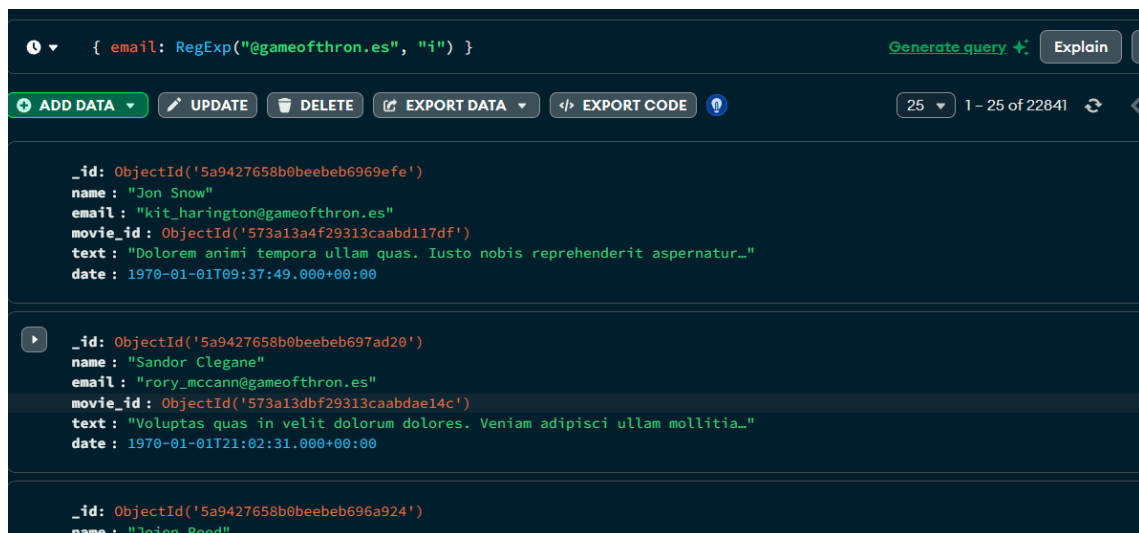
Ejercicio 1

Cuenta cuántos comentarios escribe un usuario/a que utiliza "GAMEOFTHRON.ES" como dominio de correo electrónico.

Para resolver este ejercicio usaremos **\$regex**, que se utilizará como un buscador de patrones dentro de un texto.

La razón, es que los correos electrónicos son textos y no sabemos exactamente como comienzan, ni si tienen mayúsculas y minúsculas, es por ello por lo que también se usará \$options: "i", para no distinguir entre unas u otras.

```
> db.comments.countDocuments({ email: { $regex: "@gameofthron.es", $options: "i" } })
< 22841
```



El patrón "@gameofthron.es" encuentra todos los emails que contengan ese dominio. La opción \$options: "i" hace la búsqueda insensible a mayúsculas y minúsculas, lo que es más robusto porque no sabemos cómo están escritos los emails en la base de datos. Es el equivalente al LIKE '%@gameofthron.es%' de SQL.

Gracias a los comentarios se pueden detectar patrones de comportamiento tanto positivos como negativos, esto ayuda a gestionar mejor las respuestas por ejemplo de un equipo en atención al cliente

Ejercicio 2

¿Cuántos cines hay en cada código postal situados dentro del estado Washington D. C. (DC)?

Se usa **Aggregation Pipeline**, que es como encadenar pasos uno detrás de otro.

Primero se necesita filtrar solo los cines de Washington D.C., para ello se usa **\$match**.

Después, se agrupa por código postal y cuenta cuantos hay en cada uno. Se usa **\$group** a lo que equivale a un GROUP BY en SQL.

```
> db.theaters.aggregate([
  { $match: { "location.address.state": "DC" } },
  { $group: { _id: "$location.address.zipcode", totalCines: { $sum: 1 } } }
])
< {
  _id: '20010',
  totalCines: 1
}
{
  _id: '20002',
  totalCines: 1
}
{
  _id: '20016',
  totalCines: 1
}
```

Conocer la distribución de cines por código postal dentro de un estado permite al equipo comercial planificar acuerdos de distribución geográfica.

En este estado, se encuentra una gran oportunidad, hay poca competencia, por lo que se pueden hacer mas negocios o programaciones especiales.

Nivel 3

Ejercicio 1

Encuentra todas las películas dirigidas por John Landis con una puntuación IMDb (Internet Movie Database) de entre 7,5 y 8.

```
> db.movies.countDocuments({ directors: "John Landis", "imdb.rating": { $gte: 7.5, $lte: 8 } })
< 4
```

The screenshot shows the MongoDB Compass interface. At the top, a query is entered: `{ directors: "John Landis", "imdb.rating": { $gte: 7.5, $lte: 8 } }`. Below the query bar, there are buttons for **ADD DATA**, **UPDATE**, **DELETE**, **EXPORT DATA**, and **EXPORT CODE**. The results pane shows two documents. The first document is for the movie "Animal House" (1978), and the second is for "The Blues Brothers" (1978). Both movies have an IMDb rating between 7.5 and 8.

```
db.movies.find({ directors: "John Landis", "imdb.rating": { $gte: 7.5, $lte: 8 } }, { title: 1, "imdb.rating": 1, _id: 0 })
{
  imdb: {
    rating: 7.6
  },
  title: 'Animal House'
}
{
  title: 'The Blues Brothers',
  imdb: {
    rating: 7.9
  }
}
{
  imdb: {
    rating: 7.6
  },
  title: 'An American Werewolf in London'
}
{
  title: 'Trading Places',
  imdb: {
    rating: 7.5
  }
}
```

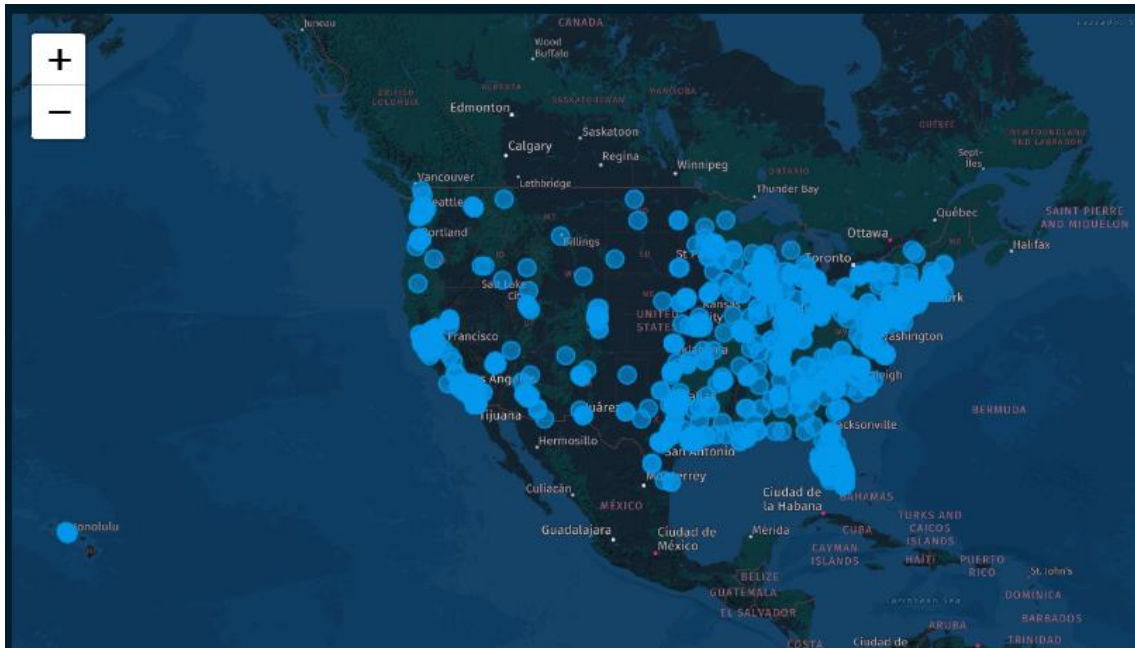
En esta consulta se usan dos filtros que tienen que cumplirse a la vez. Primero se busca que el director sea **John Landis**, y después se filtra la puntuación de IMDb para que esté entre **7,5 y 8**.

Para entrar dentro del campo de la puntuación se usa "imdb.rating", porque está dentro del objeto imdb. También se usan \$gte y \$lte, que sirven para indicar el mínimo y el máximo del rango.

Al final de la consulta se indica que solo se muestren el título y la puntuación, para que el resultado sea más claro y no aparezcan todos los datos de cada película.

Ejercicio 2

Muestra en un mapa la ubicación de todos los teatros de la base de datos.



En este ejercicio no se realiza una consulta como en los anteriores, sino que se utiliza la opción de mapa que tiene MongoDB Compass. Como la colección de teatros tiene guardadas las coordenadas en el campo `location.geo`, Compass puede reconocer esos datos y mostrarlos directamente en un mapa. De esta forma, los teatros aparecen como puntos azules y resulta más fácil comprobar dónde están ubicados.

Principalmente, la mayoría se encuentra ubicados en la costa este de los Estados Unidos, con alta densidad en Nueva York y menor presencia en el centro del país.